SVVVL
/Research

# A No-Nonsense Walkthrough of How a Causal Language Model Thinks

*The forward pass of a decoder-only transformer, one step at a time*

J. E. Whitehead III[1]

June 14, 2026

> *"What I cannot create, I do not understand."*
> — Richard Feynman

*A causal language model turns a list of tokens into a guess for the next one. That is the whole job. Everything between the input and the guess is arithmetic you can follow by hand. This document follows it. We pick one tiny, fully specified model — a nano-GPT with about eighty-five thousand numbers in it, whose only skill is sorting six letters into order — and push a single input through it from the first lookup to the last probability. Every arrow is named, every matrix has a shape, and every step is the same step the giant models take, only smaller. By the end you should be able to redraw the machine from memory.*

[1]SVVVL, Towson, MD, United States. Contact: james.whitehead@svvvl.com. See Disclosures at the end of this document.

## How to read this

Most explanations of language models reach for an analogy and stop there. The machine, we are told, "pays attention" or "predicts the next word." True, and useless: you cannot rebuild a thing from a verb. This walkthrough takes the opposite stance. We will be relentlessly concrete. There is exactly one model in these pages, it has exactly one job, and we will trace exactly one input through it.

*The Feynman test on the cover is the standard we hold ourselves to. If you cannot draw the box and label its arrows, you have not understood the box.*

The plan is simple. First we meet the model and its task. Then we walk the forward pass in the order the data flows: *embedding*, *layer norm*, *self-attention*, *projection*, *the MLP*, and how those snap together into a *transformer block*. We stack a few blocks, read off a *probability distribution* over the vocabulary, and *decode* an actual next token. Finally we explain the one trick — the *KV cache* — that makes this fast enough to be worth doing.

*This ordering is borrowed from Bycroft's interactive visualization [5], which is the best way to see what we here write down.*

Three habits will help. **Watch the shapes.** Every quantity is an array, and the only way to misunderstand a transformer is to lose track of which axis is which. We print the shape of everything. **Trust the residual stream.** There is one central object — a stack of $T$ vectors, each of width $C$ — and almost every operation reads from it and writes back to it. Keep your eye on that stack. **Believe the smallness.** The model here is so small you could, in principle, do its arithmetic on paper. That is the point. GPT-3 is the same diagram with bigger numbers.

## The model we will follow

Meet **nano-GPT**. Its entire purpose in life is to sort. You hand it a short sequence of letters drawn from the alphabet $\{\texttt{A}, \texttt{B}, \texttt{C}\}$, and it emits the same letters in sorted order. Give it `B C A B A C` and the right answer is `A A B B C C`. That is the whole task. It is a deliberately stupid job, chosen so that nothing about the *task* distracts from the *machinery*.

*This is the toy from Karpathy's minGPT [4], the same one Bycroft animates. We did not invent it; we borrow it because it is small enough to hold in your head.*

A language model does not see letters. It sees **token ids** — integers. Our vocabulary has $V = 3$ symbols, so we map

$$\texttt{A} \mapsto 0, \qquad \texttt{B} \mapsto 1, \qquad \texttt{C} \mapsto 2.$$

The model reads a context of at most $T = 6$ tokens at a time. Internally it represents each token as a vector of width $C = 48$ — the *channels*, or the width of the *residual stream*. It has $n_h = 3$ attention heads, so each head works in a slice of width $d_h = C/n_h = 16$. It stacks $L = 3$ transformer blocks. Add it all up and there are about $85{,}000$ trainable numbers. Table 1 at the end shows how those same dials read on GPT-2 and GPT-3; the architecture does not change, only the sizes.

*$T$ is the* context length*: the longest input the position machinery is built for. Bigger models stretch this to thousands or millions; the idea is identical.*

Two warnings before we start, both about what inference is *not*. First, we are watch-

***Inference vs. training.*** *Everything here is the forward pass with the weights already fixed. How those $85{,}000$ numbers got their values — gradient descent over examples — is a different story we do not*

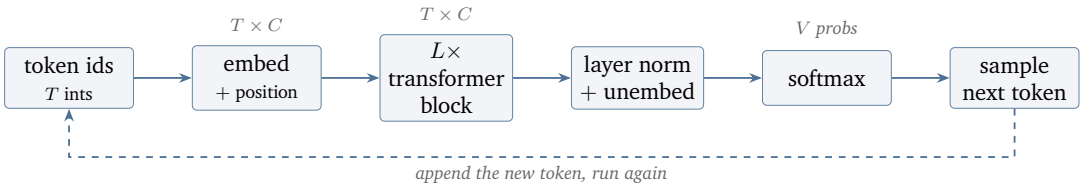


**Figure 1:** The whole machine. Token ids become vectors, flow through $L$ identical transformer blocks while keeping their shape, get turned into a probability over the vocabulary, and one token is drawn. The dashed arrow is autoregression: append the new token and run the whole thing again.

ing a trained model *run*, not learn. Every matrix below is a fixed table of numbers; nothing changes as data flows through. Second, the model is **causal**: when it forms its representation of the token at position $t$, it is allowed to look at positions $1, \ldots, t$ but never at positions after $t$. That one rule — the past may inform the present, the future may not — is what makes the model a *causal* (or *autoregressive*) language model, and it shows up concretely as a triangle of forbidden entries in Section 5.

Figure 1 is the entire document on one line. Each of the boxes gets its own section below. Notice the shape annotations: from the embedding to the final layer norm, the data is always a $T \times C$ block — six rows, forty-eight columns. The blocks reshape nothing; they only *rewrite* that block in place.

*This is why people call it the* residual stream*: a fixed-width river of $C$ numbers per position that each block edits and passes on.*

## Embedding: from integers to vectors

**Intuition.** The model cannot multiply a letter. So the very first thing it does is replace each token id with a learned vector — a row pulled from a big lookup table. A second table adds in *where* the token sits. The sum is the token's starting point in the residual stream.

**Mechanics.** Two matrices do this. The token-embedding matrix $W_E \in \mathbb{R}^{V \times C}$ has one row per vocabulary symbol; the positional-embedding matrix $W_P \in \mathbb{R}^{T \times C}$ has one row per slot in the context. For a token with id $u$ sitting at position $t$, the input vector is the row for the symbol plus the row for the slot:

$$\mathbf{x}_t \;=\; W_E[u] \;+\; W_P[t] \;\in\; \mathbb{R}^C. \tag{1}$$

*"Embedding" just means* lookup. $W_E[\,\texttt{id}\,]$ *is row number* `id`*. No multiplication happens here, only a table read. A one-hot times $W_E$ is the same thing dressed up.*

Stack these for all $T$ positions and you have the initial residual stream $X \in \mathbb{R}^{T \times C}$ — the block that will flow through every later section.

Why add the position at all? Because attention, the next big operation, is *permutation-blind*: on its own it would treat `A B` and `B A` as the same bag of tokens. The positional rows break that symmetry, stamping each vector with its slot so the model can tell first from last.

*This matters a lot for a* sorting *model. "Which letters, in which slots" is the entire input; strip the slots and the task is impossible.*

**nano-GPT instance.** Our input is `B C A B A C`, i.e. the ids $(1, 2, 0, 1, 0, 2)$ in the six slots.

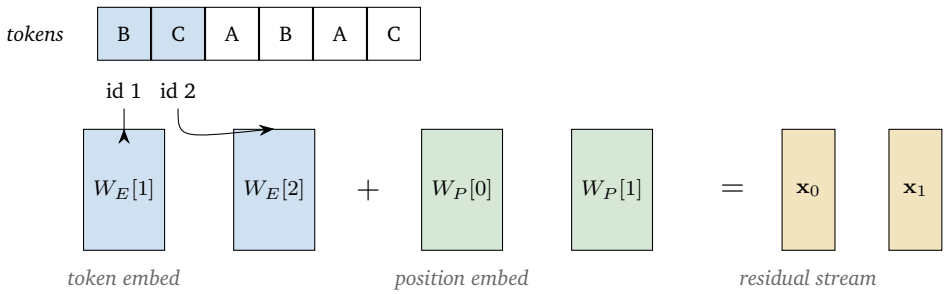


**Figure 2:** Embedding the first two slots of `BCABAC`. The token id selects a row of $W_E$; the slot index selects a row of $W_P$; their sum $\mathbf{x}_t = W_E[u] + W_P[t]$ is the starting vector (width $C = 48$) for that position. Repeat for all six slots to build the $6 \times 48$ block $X$.

Position $0$ holds `B`, so its starting vector is $W_E[1] + W_P[0]$, a specific point in $\mathbb{R}^{48}$. Do this for all six slots and $X$ is a $6 \times 48$ block of numbers. Figure 2 shows the lookup for the first two slots.

## Layer norm: keeping the numbers civilized

**Intuition.** As vectors pass through many layers, their entries can drift large or small, and that wrecks the arithmetic downstream. Layer norm rinses each vector back to a clean scale — mean zero, unit spread — before the expensive operations read it.

*Think of it as resetting the volume knob before each instrument plays, so no single channel drowns the rest.*

**Mechanics.** Layer norm acts on *one vector at a time*, across its $C$ channels. For a residual vector $\mathbf{x} \in \mathbb{R}^C$ with entries $x_1, \ldots, x_C$, compute its own mean and variance,

$$\mu = \frac{1}{C}\sum_{i=1}^{C} x_i, \qquad \sigma^2 = \frac{1}{C}\sum_{i=1}^{C}(x_i - \mu)^2,$$

standardize, then apply a learned per-channel scale $\gamma$ and shift $\beta$:

$$\mathrm{LayerNorm}(\mathbf{x})_i \;=\; \gamma_i \,\frac{x_i - \mu}{\sqrt{\sigma^2 + \varepsilon}} \;+\; \beta_i. \tag{2}$$

The tiny $\varepsilon$ just prevents a divide-by-zero. The vectors $\gamma, \beta \in \mathbb{R}^C$ are learned, which lets the model undo the standardization where it wants to; the normalization sets a sane baseline, $\gamma, \beta$ keep it expressive.

*Crucially, the mean and variance are computed* per token, per position — *across the $C$ channels, not across the $T$ positions. Each of the six vectors is normalized on its own, so the causal rule is never violated.*

**nano-GPT instance.** Each of our six $48$-vectors gets its own $\mu$ and $\sigma$. Suppose one slot's vector has drifted to a mean of $0.7$ with a spread of $2.3$; layer norm recenters it to mean $0$, rescales to spread $1$, then lets the learned $\gamma, \beta$ nudge it. The shape is untouched — in stays $6 \times 48$, out stays $6 \times 48$. In this architecture a layer norm sits just *before* each attention and each MLP (the "pre-norm" arrangement), which is why it appears twice inside every block in Section 7.

## Self-attention: letting positions talk

This is the heart of the machine, so we go slow.

**Intuition.** Up to now each position has been processed in isolation. Attention is the one operation where positions *communicate*. Every position broadcasts a question, every position broadcasts a label, and every position offers a payload. A position then gathers payloads from the others in proportion to how well its question matches their labels. For our sorting task you can imagine a position thinking "I need to know how many `A`s came before me" and pulling hardest on the positions holding `A`s.

*The standard names:* query *= the question I am asking,* key *= the label you advertise,* value *= the content you hand over if I pick you.*

**Mechanics, step by step.** Work inside a single head; the head sees vectors of width $d_h$. From each normalized residual vector we form three projections using learned matrices $W_Q, W_K, W_V$:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V, \qquad Q, K, V \in \mathbb{R}^{T \times d_h}. \tag{3}$$

*Step 1 — scores.* Compare every query with every key by dot product. The score of position $t$ attending to position $s$ is $Q_t \cdot K_s$. Collect them into a $T \times T$ matrix and scale:

$$S = \frac{QK^\top}{\sqrt{d_h}} \in \mathbb{R}^{T \times T}. \tag{4}$$

The $\sqrt{d_h}$ keeps the dot products from growing with width, which would otherwise push the next step into saturation.

*Without the $1/\sqrt{d_h}$ scaling, large dot products make the softmax nearly one-hot too early, and gradients (at training time) vanish. It is a small fix with a big effect.*

*Step 2 — the causal mask.* Here is where "causal" becomes arithmetic. A query at position $t$ must not see keys at positions $s > t$. So before normalizing, set every above-diagonal score to $-\infty$:

$$S'_{t,s} = \begin{cases} S_{t,s} & s \le t, \\ -\infty & s > t. \end{cases} \tag{5}$$

Figure 3 draws this triangle.

*Why $-\infty$ and not $0$? Because the next step exponentiates. $e^{-\infty} = 0$, so a masked position gets* exactly *zero weight. A masked score of $0$ would instead leak weight $e^0 = 1$.*

*Step 3 — softmax to weights.* Turn each row of scores into a probability distribution over the allowed positions:

$$A_{t,s} = \mathrm{softmax}_s(S'_{t,\cdot}) = \frac{e^{S'_{t,s}}}{\sum_{s' \le t} e^{S'_{t,s'}}}. \tag{6}$$

Each row of $A$ now sums to $1$ and is zero above the diagonal: row $t$ says how much position $t$ borrows from each position at or before it.

*Step 4 — gather the values.* Mix the value vectors by those weights:

$$\mathrm{head}(X) = AV \in \mathbb{R}^{T \times d_h}. \tag{7}$$

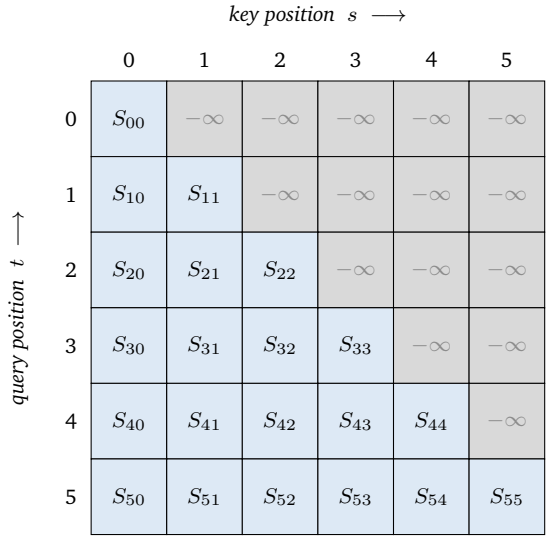


**Figure 3:** The causal mask on the $6 \times 6$ score matrix $S$ of Eq. (4). Row $t$ (a query) may attend only to columns $s \le t$ (blue); the upper triangle ($s > t$, gray) is set to $-\infty$ so that after the softmax of Eq. (6) those positions receive exactly zero weight. The future cannot inform the past.

Position $t$'s output is a weighted average of the payloads it was allowed to see.

**Multiple heads.** The model does all of this $n_h = 3$ times in parallel, each head with its own $W_Q, W_K, W_V$ and its own slice of width $d_h$. The three head outputs are concatenated back to width $C$, ready for the projection in Section 6.

*Different heads learn different "questions." One might track counts of `A`, another the most recent letter. Splitting into heads lets the model ask several questions at once without them interfering.*

**nano-GPT instance.** With $T = 6$, each head's score matrix is $6 \times 6$ and, after masking, has $1+2+3+4+5+6 = 21$ live entries out of 36. The first row attends only to itself; the last row may attend to all six. Each head emits a $6 \times 16$ block, the three stack to $6 \times 48$, and we move on.

## Projection: stitching the heads back together

**Intuition.** Three heads went off and gathered information separately. The projection mixes their findings back into a single vector per position and decides how that mixed result should be written into the residual stream.

**Mechanics.** Concatenate the head outputs into a $T \times C$ block and multiply by the output matrix $W_O \in \mathbb{R}^{C \times C}$:

$$\mathrm{Attn}(X) = \left[\, \mathrm{head}_1 \,\|\, \cdots \,\|\, \mathrm{head}_{n_h} \,\right] W_O. \tag{8}$$

Then comes the move that defines the architecture — the **residual add**. The attention result is not the new stream; it is an *edit* added onto the old one:

$$X \;\leftarrow\; X + \mathrm{Attn}(\mathrm{LayerNorm}(X)). \tag{9}$$

Read Eq. (9) carefully: layer norm feeds attention, attention is added back, and the un-normalized original $X$ survives on the left. The stream is never overwritten, only nudged.

*The residual connection is why these models train at all when stacked deep: each block only has to learn a* correction *to the stream, and the original signal has a clear, unobstructed path forward.*

**nano-GPT instance.** The concatenated heads form a $6 \times 48$ block, $W_O$ is $48 \times 48$, the product is $6 \times 48$, and it is added onto the $6 \times 48$ stream that entered the block. Shape in, shape out: $6 \times 48$.

## The MLP: thinking position by position

**Intuition.** Attention moved information *between* positions. The MLP now processes each position *on its own*, giving the model room to compute something nonlinear with what it just gathered.

*Rough division of labor: attention* routes *information across positions; the MLP* transforms *it in place. Most of a transformer's parameters live here.*

**Mechanics.** For each position's vector, expand to a wider hidden layer, apply a nonlinearity, and contract back. With GELU as the nonlinearity and hidden width $4C$ (the usual choice),

$$\mathrm{MLP}(\mathbf{x}) = \big(\,\mathrm{GELU}(\mathbf{x}W_1 + b_1)\big)\,W_2 + b_2, \qquad W_1 \in \mathbb{R}^{C\times 4C},\ W_2 \in \mathbb{R}^{4C\times C}. \tag{10}$$

The same $W_1, W_2$ are applied to every position independently — there is no mixing across slots here, so the causal rule is automatically safe. As with attention, the result is added back through a residual connection, after its own layer norm:

*GELU is a smooth cousin of the ReLU: it lets a little negative signal through near zero instead of hard-clipping it [7].*

$$X \;\leftarrow\; X + \mathrm{MLP}(\mathrm{LayerNorm}(X)). \tag{11}$$

**nano-GPT instance.** Width in is $48$, the hidden layer is $4 \times 48 = 192$, width out is $48$. Each of the six positions runs through the same little two-layer network independently, then writes its correction back into the stream. Shape in, shape out: $6 \times 48$.

## The transformer block, assembled

We now have all the parts. A **transformer block** is exactly the two residual updates we just wrote, in order: normalize, attend, add; normalize, MLP, add.

$$X \;\leftarrow\; X + \mathrm{Attn}\big(\,\mathrm{LayerNorm}(X)\big), \tag{12}$$

$$X \;\leftarrow\; X + \mathrm{MLP}\big(\,\mathrm{LayerNorm}(X)\big). \tag{13}$$

Figure 4 draws it. The block takes a $T \times C$ stream and returns a $T \times C$ stream — same shape — which is precisely why blocks can be stacked without anything in between.

*Note the two layer norms sit* inside *the residual paths, before attention and before the MLP. This "pre-norm" layout is what almost every modern model uses; it makes deep stacks behave.*

**nano-GPT instance.** Our model stacks $L = 3$ of these. The $6 \times 48$ stream enters block 1, comes out $6 \times 48$, enters block 2, and so on. Three rounds of "route, then

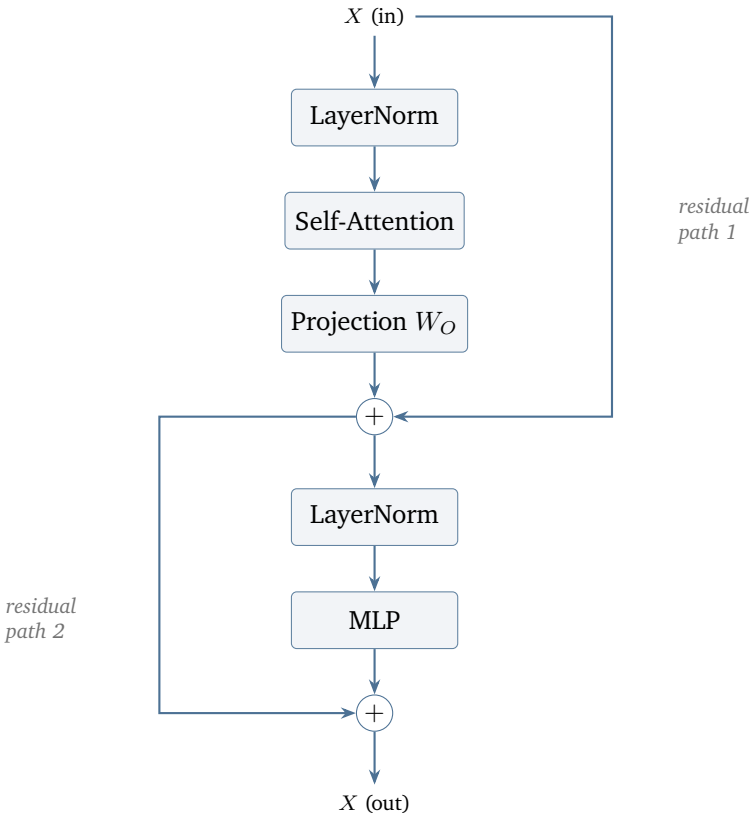


**Figure 4:** One transformer block, Eqs. (12)–(13). The stream is normalized, passed through attention and its projection, and added back (residual path 1); then normalized again, passed through the MLP, and added back (residual path 2). Shape is preserved end to end, so blocks stack directly.

transform," each writing its small corrections into the same river of numbers.

## Unembedding and softmax: from vectors to a distribution

**Intuition.** After the last block, each position holds a rich $48$-vector. To make a prediction we must turn the *last* position's vector into a score for every vocabulary symbol, then into probabilities.

*Why the last position? Because in a causal model, position $t$'s output vector is the model's summary of "everything up to and including $t$" — exactly the context from which the* next *token should be predicted.*

**Mechanics.** Apply one final layer norm, then multiply by an unembedding matrix to get **logits** — one raw score per vocabulary symbol:

$$\ell = \mathrm{LayerNorm}(\mathbf{x}_T)\, W_U \in \mathbb{R}^V, \qquad W_U \in \mathbb{R}^{C\times V}. \tag{14}$$

Many models tie $W_U = W_E^\top$ — the same table that read tokens in is reused to read them out. The logits are then squashed into a probability distribution by the softmax:

$$p_k = \mathrm{softmax}(\ell)_k = \frac{e^{\ell_k}}{\sum_{j=1}^{V} e^{\ell_j}}, \qquad \sum_k p_k = 1. \tag{15}$$

*This "weight tying" saves parameters and reflects a pleasing symmetry: the geometry that turns symbols into vectors is reused to turn vectors back into symbols.*

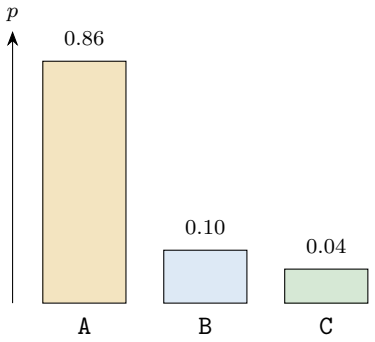


**Figure 5:** The output distribution from Eq. (15) for the first predicted slot of `B C A B A C`. The model concentrates probability on `A`, the correct smallest letter. The numbers are illustrative.

**nano-GPT instance.** The last position's 48-vector hits $W_U$ ($48 \times 3$) and produces three logits, one each for `A`,`B`, `C`. Softmax turns them into three probabilities. For our input `B C A B A C`, a well-trained model puts almost all the mass on `A` for the first output slot, because `A` is the smallest letter and must come first. Figure 5 sketches that distribution.

## Output: turning a distribution into a token

**Intuition.** A distribution is not yet an answer. *Decoding* is the rule that picks one concrete token from the probabilities — and the same logits can yield different behavior depending on the rule.

*This is the one place the model's output is genuinely a* choice*. Everything before Eq. (15) is deterministic arithmetic; sampling is where randomness, if any, enters.*

**The common rules.**

- **Greedy.** Take the most probable token, $\arg\max_k p_k$. Deterministic; for our sorting model, the right rule — there is one correct next letter.
- **Temperature.** Before the softmax, divide logits by a temperature $\tau$: $p_k \propto e^{\ell_k/\tau}$. Low $\tau$ is conservative, high $\tau$ is adventurous.

  *$\tau < 1$ sharpens the distribution toward greedy; $\tau > 1$ flattens it toward uniform; $\tau \to 0$ is greedy.*
- **Top-$k$ / top-$p$.** Keep only the $k$ most probable tokens (or the smallest set whose mass exceeds $p$), renormalize, and sample from those. Cuts off the long tail of bad tokens while keeping some variety.

**The autoregressive loop.** Whatever the rule, the chosen token is *appended* to the input, and the entire forward pass runs again to predict the *next* token — the dashed arrow in Figure 1. This is the sense in which the model is autoregressive: its own output becomes its next input.

**nano-GPT instance.** Greedy decoding on Figure 5 emits `A`. We append it, rerun, and the model — now conditioned on the input plus its own `A` — emits the second sorted letter, and so on until the six sorted letters `A A B B C C` are complete.

## The KV cache: why you do not redo all the work

There is a glaring waste in the loop just described. To predict token $t+1$ we rerun the whole forward pass on tokens $1, \ldots, t$. But at the previous step we already ran it on $1, \ldots, t-1$. Almost everything is being recomputed.

**The fix.** Look back at attention. For a *new* query at position $t$, the scores it needs are $Q_t \cdot K_s$ and the payloads $V_s$ for all $s \leq t$. The keys and values $K_s, V_s$ for the *old* positions $s < t$ do not depend on the new token at all — they were already computed. So we keep them. The **KV cache** stores the key and value vectors of every position seen so far. At each new step the model computes $K, V$ for only the *one* new token, appends them to the cache, and attends against the whole cache.

*This works* because *the model is causal: old positions never attend to the new one, so adding a token cannot change any past $K$ or $V$. Causality is what makes the cache correct.*

**What changes.** Without the cache, generating $n$ tokens costs work growing like $n^2$ in the sequence length — you reprocess the whole prefix every step. With it, each step does work proportional to the current length, and the per-step cost of re-embedding and re-running the prefix disappears. The arithmetic of any single step is exactly what we wrote in Sections 3–9; the cache only avoids *repeating* the steps for tokens whose representations are already settled.

*The cost moves from compute to memory: the cache itself grows with the sequence length and the number of layers, and for long contexts it, not the arithmetic, becomes the bottleneck.*

## Putting it together

Trace the whole machine once more, end to end, on `B C A B A C`. Six letters become six ids; each id selects a row of $W_E$ and adds a row of $W_P$, giving a $6 \times 48$ residual stream (Eq. (1)). That stream flows through three transformer blocks; in each, a layer norm feeds self-attention, whose causal mask lets every position pool information only from its past (Figure 3), the heads are stitched by a projection and added back, and then a per-position MLP refines each vector and is added back too (Figure 4). After the third block, the last position's vector is normalized, multiplied by the unembedding to three logits, and softmaxed into a distribution over `A`,`B`,`C` (Eq. (15)). Greedy decoding reads off `A`; we append it and run again. Six passes later the model has written `A A B B C C`.

Every one of those operations was small enough to check by hand. Now read Table 1: the same operations, the same diagram, run inside GPT-2 and GPT-3. Nothing in the wiring changes between the toy and the giant — only $C$, the head count, the layer count, and the context length grow. That is the one idea worth keeping: *a frontier language model is this walkthrough, scaled.*

**Table 1:** The same dials, three sizes. The architecture in Figures 1–4 is identical across all three; only the numbers grow. Figures are the standard published configurations.

| **Quantity** | **nano-GPT** | **GPT-2 (small)** | **GPT-2 (XL)** | **GPT-3** |
|---|---|---|---|---|
| Vocabulary $V$ | 3 | 50,257 | 50,257 | 50,257 |
| Context length $T$ | 6 | 1,024 | 1,024 | 2,048 |
| Channels $C$ | 48 | 768 | 1,600 | 12,288 |
| Heads $n_h$ | 3 | 12 | 25 | 96 |
| Blocks $L$ | 3 | 12 | 48 | 96 |
| Parameters | $\sim$85k | 124 M | 1.5 B | 175 B |

## Further Reading and References


[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). "Attention Is All You Need." *Advances in Neural Information Processing Systems*, 30.

[2] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). "Language Models Are Unsupervised Multitask Learners." *OpenAI Technical Report* (GPT-2).

[3] Brown, T. B., Mann, B., Ryder, N., et al. (2020). "Language Models Are Few-Shot Learners." *Advances in Neural Information Processing Systems*, 33 (GPT-3).

[4] Karpathy, A. (2020–2023). *minGPT* and *nanoGPT*: minimal PyTorch re-implementations of GPT. `https://github.com/karpathy/minGPT`.

[5] Bycroft, B. (2023). *LLM Visualization* — an interactive 3D walkthrough of a GPT-style model. `https://bbycroft.net/llm`.

[6] Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). "Layer Normalization." *arXiv:1607.06450*.

[7] Hendrycks, D., & Gimpel, K. (2016). "Gaussian Error Linear Units (GELUs)." *arXiv:1606.08415*.

## Disclosures

The views expressed in this document are those of the author and SVVVL alone and do not constitute investment, legal, tax, or accounting advice. This document is provided for informational and research purposes only and is not an offer or solicitation to buy or sell any security or financial product.

The framework presented is theoretical and relies on idealized assumptions that do not hold in practice. The results should not be implemented in a real portfolio without additional constraints and considerations not addressed in this paper.

While reasonable care has been taken in the preparation of this document, no representation or warranty, express or implied, is made as to its accuracy or completeness.

Reading this document does not create an advisory, fiduciary, or client relationship of any kind between the reader and SVVVL or the author.